- *Rules to be verified and validated, to be applied to data and functions* --None in this use case
- *Transactions to be recorded in the digital ledger* -- `initialize()`, `increment()`, and `decrement()`

Note that you may decide to record only the functions or transactions that change the value of the counter. Thus, not all functions specified in the smart contract will result in transactions to be saved in the blockchain's distributed ledger. The `get()` function is to view the contents of the counter, and you may not want its invocation recorded on the blockchain. You can specify this characteristic by defining it as a view-only function. The transactions of view functions are not recorded on the blockchain.

2.2.3 From class diagram to contract diagram

In this step of the design process, you'll define the UML class diagram as a guide for the design of the solution to the counter problem. A class diagram defines the various structural elements of the solution. It draws upon the items discovered in the previous two steps (creating the use case diagram and digital asset analysis). The typical UML class diagram of traditional object-oriented programming (OOP) shown on the left side of figure 2.4 has three components:

- Name of the class
- Data definitions
- Function definitions